A New Era of OCaml Editing
Powered by Merlin, Delivered via LSP

Xavier Van de Woestyne
xavier@tarides.com
Tarides

Sonja Heinze
sonja@tarides.com
Tarides

Ulysse Gérard
ulysse@tarides.com
Tarides

Muluh Godson
pizie@tarides.com
Tarides

July 2025

Abstract

In an era where the number of code editors keeps growing, supporting only `Vim` and `Emacs` is no longer enough for a modern language like OCaml. Maintaining a proliferation of dedicated editor clients has become a real burden — what we informally call the *editor-mode burnout*. Fortunately, *Language Server Protocol* (`LSP`)[1], which was originally introduced in 2016 for the editor `Visual Studio Code` (`VSCode`), offers a generic open standard for all editors. In this presentation, we will present our work on adopting `LSP`. We will explain how it has significantly improved OCaml editor support while keeping maintenance efforts to a minimum — we'll also explain the costs it comes with.

1 The Age of Merlin

For many years, the standard way of writing OCaml involved combining a popular text editor, often `Emacs` or `Vim`, with various enhancements: a plugin for syntax highlighting, the external tool `ocp-indent` for indentation and cursor placement, and `Merlin`, created by Frédéric Bour and Thomas Refis. `Merlin`, which runs as a background process, is an editor-agnostic server capable of providing *IDE-like services* for OCaml. It provides contextual information about the code, enabling a conventional editor to compete with the fully integrated IDEs commonly associated with platforms such as `.NET` or the `JVM`, examples include `Eclipse` and `Visual Studio`. Over the years, `Merlin` users have expressed great enthusiasm for the tool, often describing it as forward-thinking, efficient, and genuinely helpful, but maintaining multiple clients can lead to *editor-mode burnout*[1], as pointed out by its creators.

[1]We adopt the terminology used in the Merlin experience report[2], in which *editor-mode burnout* denotes the fatigue and maintenance burden associated with supporting multiple editors.

1.1 Editor-Mode Burnout

We call *editor-mode burnout* the phenomenon that encompasses a range of frustrations related to the maintenance of editor support: high maintenance burden, understaffed tooling efforts, lack of visibility or credit and huge dependency on *potentially missing* volunteers. Although all these points are closely intertwined, we will focus primarily on the first issue — the high maintenance cost — as it is relatively straightforward to illustrate, and because `LSP` provides a fairly direct solution to this particular problem.

While `Merlin`'s editor-agnostic design is one of its key strengths, it also comes with a significant maintenance cost. Each new code editor — which nowadays emerge rapidly (`NeoVim`, `Zed`, `Helix`), each with distinct interaction paradigms — requires its own integration layer to leverage Merlin's capabilities. This often entails substantial bootstrapping work, which can hinder the adoption of OCaml in modern development environments.

There are several approaches to mitigate editor-mode burnout, such as delegating the support of community plugins to the community itself (*a strategy that has been used within the OCaml community for several years*) or moving toward standardizing IDE interactions, as proposed by the *Language Server Protocol*.

We will focus on the gradual migration to `LSP` as the primary interaction server, aiming to *drastically* reduce the maintenance burden of complex editor plugins.

2 LSP: A Protocol to Rule Them All

Historically developed as part of `Visual Studio Code`, the `LSP` is an open protocol[1], based on `JSON-RPC`[3], that standardizes the interactions between an editor and a server aiming to provide *IDE-like services*. The protocol defines a collection of standardized features, generic across programming languages, organized around five main pillars:

- **Language intelligence**: this relates to the set of possible interactions when hovering over a term, *go to definition/implementation/declaration*, finding occurrences and hierarchical navigation within the outlines of a document or workspace.
- **Code assistance**: including completion, Signature help for displaying function type while writing, semantic syntax highlighting, code actions, occurences highlight and renaming.
- **Code quality**: including diagnostics, code-lens and formatting.
- **Synchronization**: offers a standard way of keeping the server synchronised with the active documents and offering a customisation perspective, at user level, by workspace or project allowing partial implementation of the protocol and progressive upgrades.
- **Project awareness**: workspace management and the ability to react to file management events like add/delete/edit/rename.

The protocol defines a wide variety of interactions and has seen an impressive adoption: the editor `VSCode`, which is natively based on `LSP`, is used by 73.6% of people responding to the StackOverflow Survey in 2024[4]. Furthermore, `LSP` has become a standard protocol across editors, including older editors like Vim and Emacs via an external package, `lsp-mode` or directly bundled, via `eglot`.

Viewed through this lens, providing an `LSP` server for OCaml seemed like the holy grail for supporting any editor compatible with LSP. However, even though the protocol is very rich, the attempt to standardize it *to support all languages* naturally results in losing some features specific to OCaml editing[2], and sometimes the protocol assumes, somewhat aggressively, that all editors interact via a mouse and without any particular modality.

It's also impressive how excellent `LSP`'s *defaults* are. Indeed, without custom extensions, a *full-featured* server provides a relatively good and rich experience, reducing the pressure of supporting every editor over the years. It is not realistic to ask users to migrate from one server to another, for example from `Merlin` to an `LSP` server, if the latter's support is less rich and less specific than its predecessor. In our case, `Merlin` offers strictly more features relevant to OCaml than a bare-bones `LSP` support.

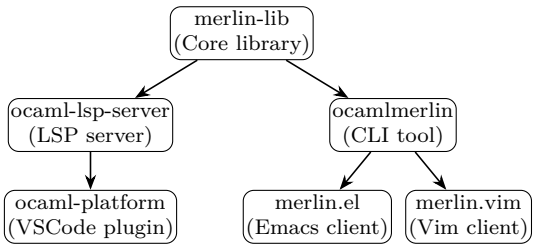
2.1 Merlin and LSP

Given the widespread adoption of `VSCode`, implementing an `LSP` server for OCaml was an essential step to reach the large share of potential OCaml users using `Visual Studio Code`. This new language server, called `OCaml-lsp`, paved the way to the realization that supporting `LSP` as the primary interaction server or OCaml could significantly reduce the maintenance burden across different clients and potentially lower the risk of *editor-mode burnout*.

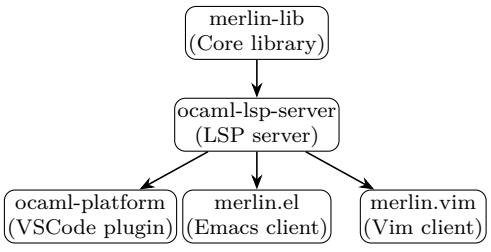
Initially, `OCaml-lsp`, initiated by Rudi Grinberg, was a server specifically designed for integration, using `Merlin` as a library, with `VSCode`, with a full `binding` to interact with `LSP`, paired with `vscode-ocaml-platform`, also initiated by Rudi Grinberg, currently maintened by Sora Morimoto, a plugin configured to interact seamlessly with `OCaml-lsp`. This increased the fragmentation of editor support officially maintained: `VSCode`, `Vim` and `Emacs`), leading to the following situation:

	Emacs	**VSCode**	**Vim**
Server	Merlin	LSP	Merlin

This leads to the following hierarchy of tools, which still involves full support for `Emacs` and `Vim`.



Keeping `merlin-lib` makes a lot of sense because it confines the complex interactions tied to each OCaml compiler update within a single package (`Merlin` effectively vendorizes the compiler frontend). However, the ideal scenario would be to deprecate `ocamlmerlin` and rely as much as possible on the *defaults* provided by `LSP`, drastically reducing the maintenance burden of additional plugins, requiring only support for OCaml-specific features. This leads us to the following lattice of relationships:



Which brings us to a question: how does `LSP` fit the common use cases for OCaml?

2.2 The need for extension

As we have seen, merely supportingLSP is not a sufficient solution, as the editing experience in OCaml involves several specificities that are rarely encountered in other languages. For example:

- `type-enclosings`: allowing the user to inspect the type of the expression under the cursor, adjust the

[2]Features such as `destruct`, `construct`, enclosing type, polarity-based search, and many others that have made the Vim and Emacs user experience for OCaml so successful.

level of verbosity (e.g., by expanding type aliases), and increase or decrease the scope of the enclosing expression being observed.

- `destruct`: allowing the user to generate missing patterns in a pattern matching expression, or to construct a complete match expression capable of handling a given value.
- `construct`: allowing the user to select a type-relevant expression to fill a typed hole — particularly effective when used in combination with `destruct`, which may introduce such holes.
- `navigation between holes`: allowing the user to navigate between typed holes, whether defined manually or introduced by the `destruct` command.
- `search`: allowing the user to search for values by type or polarity.

This set of features is not supported by the protocol. Therefore, it is important to design client-side support to integrate them. By delegating standard `LSP` handling to the editor, we reduce the need for bootstrapping a fully custom client. At the same time, we provide extensions — including for `VSCode`[3] — to support these features and avoid limiting ourselves in the future when introducing new capabilities that may prove valuable to the community.

3 Transitioning smoothly to `LSP`

As we have seen earlier, simply removing `ocamlmerlin` would lead to a degradation of the features provided by `Merlin` (making the developer experience significantly richer in `VSCode`, which is backed by a dedicated client, compared to its alternatives). Therefore, this is not an acceptable solution.

We have identified three ways to adapt `LSP` — *as much as possible* — to our use cases: one primarily social approach, and two others of a technical nature: request adjustments to the protocol (the *social* one), using *code actions* and using *custom requests*.

We will not dwell on requesting adjustments to the protocol, as it is very hard to get them through.

Custom requests and *code actions* can be conditionally enabled through a capabilities mechanism, configurable on both the client and server sides. This flexibility supports the exploration of new features in an experimental context, enabling a controlled incubation phase prior to potential broader adoption.

3.1 Extension via *code action*

code actions are **contextual** actions that can be invoked on demand. Depending on the cursor position and its — often syntactic — context, the server returns a list of executable actions that allow manipulating the file and can, upon request, broadcast a list of commands interpretable by the editor.

Some features of `Merlin` naturally fit this approach, for example `merlin-destruct`. The syntactic context of `merlin-destruct` is *whether the cursor is positioned on a pattern*. When the cursor is on a pattern, `merlin-destruct` allows replacing the pattern with its pattern-matching cases. Furthermore, `merlin-destruct` broadcasts the command `jump-to-next-hole` to move the cursor to the next *hole*.

The typical *workflow* for using an action consists of validating its existence based on the context, invoking it, performing modifications (or movements) in the current file, and then optionally broadcasting a series of client commands.

3.1.1 Pros

The great advantage offered by *code actions* is that their interaction mechanism is sufficiently generic to capture a wide range of behaviors, allowing an action to require no additional work for all editors. This makes it a standard way to extend the usual behavior of `LSP`. Furthermore, if a client command broadcast by an action is not supported by the editor executing the action, it is silently ignored, which keeps the process unobtrusive.

3.1.2 Cons

An action is only parameterized by its context; it is not possible to interact with the user to provide further specifications. This is mainly because, in addition to abstracting interaction with a file, it would also require abstracting interaction with the user — effectively generalizing the notion of a *Widget*.

We also observed that when generating a collection of actions based on context — for example, when the cursor is on a *hole* — producing all possible *constructible expressions* can lead to a highly visually obtrusive experience.

3.2 Extension via *custom request*

An `LSP` server being a `JSON-RPC` server that is *relatively* arbitrary, it is possible to interpret arbitrary methods by assigning them a name (as long as it is not reserved by `LSP`). While these are commonly referred to as *custom requests*, this concept is not formally covered by the specification. Since each method is associated with parameters, the server can validate these custom requests and provide specific responses. Unlike code actions, the interaction here works by sending a request to the server and interpreting its response on the client/editor side.

3.2.1 Pros

The use of custom requests makes it possible to completely abstract away the potential rigidity of

[3] `VSCode` has proven to be significantly less flexible than initially expected, as its design is tightly coupled to `LSP`. This constraint has occasionally forced us to resort to somewhat inelegant workarounds in order to provide a satisfactory user experience — particularly for features such as `type-enclosings`.

the protocol, by supporting, on the editor side, the construction of rich sets of parameters. It also allows chaining multiple requests, enabling interactive editing modes. A notable example is the implementation of the `merlin-type-enclosing` command, which allows interaction with the verbosity of the displayed type — making it possible to *grow* or *shrink* the observed enclosing expression through successive calls to the request.

3.2.2 Cons

Since custom requests are not part of the official protocol specification, they are not natively supported by editors. As a result, improving the user experience through such requests requires dedicated integration work for each editor (using `LSP`).

Moreover, to ensure the long-term stability of features that introduce a new layer of indirection between the server and the editor, it is likely necessary to implement an explicit API versioning strategy. This would help facilitate the gradual migration of custom requests and avoid breaking changes. Currently, such a mechanism has not yet been implemented in our server.

3.3 A decision behind every feature

Each deviation from the protocol, as we've seen, brings its own set of strengths and trade-offs. For each feature, we ask a series of questions to determine whether it can be integrated through one or more actions, aiming to minimize editor-specific development:

- Should the feature be configured by arguments that are independent of the context?
- Does the feature require additional interaction, such as choosing one option from a set of possible results?

In addition, if the feature overly clutters the UI (for example, by returning a non-fixed set of results) or requires too much interactivity, we generally rely on one or more *custom requests* instead of a *code action*. When the choice to use an action is a fallback option, we generally add requests that can replace the action, allowing editors time to integrate the feature more ergonomically via requests. Thanks to capabilities and configuration modes, this approach enables support from the feature's release (*modulo* client command implementation) while leaving the door open for more ergonomic client support through requests.

3.3.1 A direct bridge to Merlin

To enable the incubation of features provided by Merlin, but not yet officially specified on the `OCaml-lsp-server` side, we developed a *custom request* that directly invokes Merlin's protocol — just as if it were called from the `CLI` — thereby accelerating the migration to the `LSP` protocol.

4 Practical implementation

The way Merlin and `OCaml-lsp-server` interact generally follows this flow: every feature is exposed through `Merlin`'s protocol and utilized within the implementation of the `LSP` server. If the feature is part of the `LSP` protocol, it is exposed by the server as-is. If the feature is specific to OCaml, it is integrated following the decision process outlined in the previous section.

4.1 Clients

Currently, although the `LSP` server can be used by any editor that has an `LSP` client implementation and thus benefits from good defaults out of the box, there are two fully OCaml-featured implementations whose functionalities rival those offered by the traditional Merlin clients:

- **Visual Studio Code**: mentioned multiple times, the OCaml client implementation in `VSCode` provides a range of OCaml-specific features offering a user experience comparable to using Merlin in other editors (in terms of feature support).
- **Emacs**: `ocaml-eglot` is an `LSP` client plugin for `Eglot` (the default `LSP` client bundled with `Emacs`) that offers as many—if not more—features than the venerable `merlin.el` mode (which was remarkably ahead of its time and included numerous subtle ergonomic features). Its implementation has also inspired an informal list of best practices for developing clients whose interactions rely primarily on the keyboard (like `Vim` or `Emacs`).

From our point of view, `ocaml-eglot` is an excellent example of the benefits brought by `LSP` because, for an equivalent feature experience compared to the traditional `Emacs` mode, the code is significantly shorter, simplifying its maintenance. Based on these experiences, work on a `NeoVim` plugin for OCaml-specific `LSP` support is underway.

5 Conclusion

Once past the initial frustration caused by the limitations of `LSP` in perfectly fitting a language as rich as OCaml, as soon as the right criteria for choosing between actions and requests for extensions are found, the benefits become **clearly noticeable**. We were able to greatly extend the user experience for OCaml development in `VSCode`, and we have also completed another client, `Emacs`, via `Eglot`, which is capable of rivaling its predecessor (`merlin.el`) in terms of functionality all while drastically reducing the necessary codebase, making it more maintainable and modern.

The OCaml editor support is still very active, users will continue transitioning from the venerable `ocamlmerlin` to `LSP`, thereby reducing the maintenance cost of currently supported editors as well as those we will support in the future. This opens new perspectives: indeed, after abstracting interactions through *Language Servers*, the next step — while continuing to improve the existing tools and add new features.

References

[1] Language server protocol specification, 2016.

[2] Frédéric Bour, Thomas Refis, and Gabriel Scherer. Merlin: A language server for ocaml (experience report). 2018.

[3] Json-rpc 2.0 specification, 2010.

[4] Stack overflow developer survey 2024 — technology, 2024.